
Robustness Analysis of Transformer-Based Prompt Injection Detection Against Paraphrased and Obfuscated Attacks

Iham Araf¹ Md. Saidul Islam Chowdhury¹ Sourav Biswas¹

Abstract

Prompt-injection detectors are usually evaluated on clean, English test sets drawn at random from the same pool as the training data. We ask how much of the reported accuracy survives when the surface form of a prompt changes but its intent does not. We define a test protocol around a group-aware split and six test conditions (clean, paraphrased, obfuscated, Bangla–English code-mixed, cross-dataset and a false-positive-only set), and we build SentinelPrompt-BnEn, 900 prompts expressing 300 intents in English, Bengali script and Romanized code-mix. Training on 462 English and German prompts, we compare three TF-IDF classifiers, fine-tuned DistilBERT and RoBERTa, an off-the-shelf DeBERTa-v3 detector and an adversarially augmented DistilBERT. Clean macro-F1 is 0.87–0.98 and paraphrasing costs the models we train at most 7%. Obfuscation costs the fine-tuned transformers 17–29%, not because they miss attacks (attack success rate $\leq 0.4\%$) but because obfuscated harmless prompts are flagged; TF-IDF models lose 2–5%. On Bengali-script prompts every model except the augmented one scores 0.33–0.39 macro-F1, the level of a constant predictor and the two families fail in opposite directions: transformers flag harmless Bangla (RoBERTa assigns 0.988 injection probability to the Bengali-script word for “okay”), TF-IDF models miss Bangla attacks. Augmentation raises obfuscated macro-F1 from 0.778 to 0.892 and code-mixed from 0.588 to 0.787, but does not transfer to a held-out encoding and leaves specificity on instruction-style benign prompts at 0.002. Contrary to our hypothesis, the group-aware split changes accuracy by less than 0.01 on this corpus.

¹Department of Computer Science and Engineering, United International University, Dhaka, Bangladesh. Student IDs: Iham Araf 0112310515; Md. Saidul Islam Chowdhury 0112230636; Sourav Biswas 0112230635. Supervisor: Kazi Fahim Tahmid. Correspondence to: Iham Araf <ihamaraf805@gmail.com>.

Course project report, CSE 4891 Data Mining, Department of CSE, United International University, Dhaka. October 2026.

1. Introduction

An application built on a large language model (LLM) mixes trusted instructions with untrusted text: user messages, retrieved web pages, e-mails. Prompt injection exploits that mix by placing instructions in the untrusted part, either directly (Perez & Ribeiro, 2022) or through content the application retrieves (Greshake et al., 2023). A common mitigation is a lightweight classifier that screens text before it reaches the model. Such detectors report high accuracy, but the evaluations behind those numbers tend to share three properties: the test attacks are written in the same style as the training attacks, the test set is English and the split is random.

Each property hides a different failure. An attacker can reword an attack, or perturb its characters, without changing what it asks for (Jin et al., 2020; Wei et al., 2023). Hundreds of millions of people write to chat systems in languages other than English and many of them, Bangla speakers included, mix scripts and languages within one message (Khanuja et al., 2020). A random split of a corpus that contains near-duplicates can put copies of the same prompt on both sides, so that test accuracy partly measures memory (Elangovan et al., 2021; Lewis et al., 2021).

This report takes an evaluation-driven view: the contribution is a test protocol and a test set, not a new detector. We ask five questions. **RQ1**: how far does performance fall under paraphrase? **RQ2**: how far under character- and word-level obfuscation? **RQ3**: does an English-trained detector generalise to Bangla–English prompts? **RQ4**: how much does a leakage-free split lower reported accuracy? **RQ5**: what is the accuracy–latency trade-off between classical and transformer models? Our contributions are:

- A protocol with a group-aware split and six paired or held-out test conditions, with robustness drop rate (RDR), attack success rate (ASR), exact McNemar tests and bootstrap intervals (Section 4).
- SentinelPrompt-BnEn, 900 prompts in English, Bengali script and Romanized code-mix, balanced by label within each language, with a three-annotator verification of 200 rows (Section 3.3).

- An evaluation of seven detectors which finds that obfuscation and Bengali script break fine-tuned transformers mainly through false positives, that the two model families fail in opposite directions on Bangla and that adversarial augmentation helps only for the perturbations it was shown (Section 5).
- A controlled leakage ablation which finds no measurable inflation on this corpus, and a shortcut audit that traces the Bangla false positives to the script (Sections 6 and 7).

2. Related Work

Prompt injection. Perez & Ribeiro (2022) showed that simple goal-hijacking and prompt-leaking instructions redirect GPT-3 based applications. Greshake et al. (2023) extended the threat to indirect injection through retrieved content and Liu et al. (2024) formalised attacks and defences and benchmarked them across models and tasks. Jailbreaks, which target the model’s safety training rather than the application’s instructions, overlap in technique; Wei et al. (2023) attribute their success to competing objectives and mismatched generalisation, including encodings such as Base64 that safety training did not cover. PromptBench (Zhu et al., 2023) measures how adversarial perturbations of a task prompt change an LLM’s answers; we reuse its prompts as benign, instruction-style text.

Robustness of text classifiers. Word-substitution attacks such as TextFooler (Jin et al., 2020) and behavioural test suites such as CheckList (Ribeiro et al., 2020) show that high held-out accuracy can coexist with brittle behaviour on meaning-preserving edits. We apply the same idea to injection detection, with label-preserving paraphrases checked by a sentence-embedding similarity gate (Reimers & Gurevych, 2020) and a dose–response design for character obfuscation.

Code-mixing and Bangla. GLUECoS (Khanuja et al., 2020) benchmarks code-switched NLP for Hindi–English and Spanish–English. BanglaBERT (Bhattacharjee et al., 2022) provides Bangla pretraining and benchmarks. We are not aware of a prompt-injection test set for Bangla and in particular none that holds the intent fixed while varying script, which is what lets us separate language from content.

Leakage and shortcuts. Leakage, information in training that would not be available at prediction time, can make evaluation optimistic (Kaufman et al., 2012). Train–test overlap inflates scores in NLP benchmarks (Elangovan et al., 2021; Lewis et al., 2021) and deduplication changes what models learn (Lee et al., 2022). Models also exploit annotation artefacts and other shortcuts (Gururangan et al., 2018; Geirhos et al., 2020); our audit looks for two such shortcuts, prompt length and script.

Table 1. Sources after cleaning and deduplication and the role of each. D1 is the only source used for training.

Source	Safe	Inj.	Role
D1 deepset	399	263	train / val / E1–E3
D2 Lakera	0	1,000	E5 (recall only)
D3 PromptBench	4,150	0	FP test (specificity)
D4 SentinelPrompt-BnEn	453	447	E4

3. Data

3.1. Sources

Table 1 lists the four sources. **D1**, deepset/prompt-injections (deepset, 2023), contains labelled benign and injection prompts in English and German. **D2**, Lakera’s gandalf_ignore_instructions (Lakera AI, 2023), contains crowd-sourced attempts to make a model reveal a password; every row is an injection. **D3** is the prompt collection of PromptBench (Zhu et al., 2023). Its rows are task instructions (“Classify the relationship between the provided sentences as ‘entailment’ or ‘not_entailment’.”), so every row is benign. **D4** is SentinelPrompt-BnEn (Section 3.3).

All sources are mapped to one schema (id, text, label, source, language, attack_type, variant_type, group_id, split). Pre-processing is deliberately minimal: Unicode NFKC normalisation and whitespace collapsing only. Stop words, punctuation and special characters are kept and nothing is spell-corrected, since “correcting” leetspeak would erase the signal under test. Zero-width characters are recorded as a boolean feature before normalisation. Exact duplicates are removed, which leaves 6,712 rows; near-duplicates are kept and grouped, as described next.

3.2. Near-duplicate groups and the split

Near-duplicates are found with MinHash and locality-sensitive hashing over character 5-gram shingles (Broder), Jaccard ≥ 0.80 , cross-checked against TF-IDF cosine ≥ 0.85 . Each connected component of the near-duplicate graph becomes one group_id. Outside D4, 327 groups contain more than one row, covering 1,502 of 5,812 rows (25.8%); 1,468 of those rows are PromptBench prompts. D4’s 300 intents are groups by construction. Every adversarial variant later inherits its parent’s group_id.

The split is grouped: all rows sharing a group_id land in the same partition (70/15/15 by group shuffle; stratified group folds are used for cross-validation). The share of injections is 42%, 36% and 34% in train, validation and test, against 40% in D1 overall. Two sources are held out entirely. D4 is never used for training or validation, so that RQ3 tests transfer rather than fitting. D2 is held out as the cross-dataset set and a guard function fails any step that

would place a Lakera row in training. D3 cannot measure detection at all, since it has no injections, so it is reported only as a false-positive test. D1 is therefore the only training source: 462 training rows (193 injections), 100 validation rows and 100 clean test rows, of which 34 are injections.

3.3. SentinelPrompt-BnEn (D4)

D4 holds 300 intents, each in three versions: English (en), Bengali script (bn) and Romanized Bangla mixed with English (bn-en). The three versions share a `group_id`, so any difference in a model’s output between them is due to language and script, not content. Half of the intents are injections, 25 for each of six attack types (direct override, role play, encoding, context switch, payload splitting, indirect); the other half are benign and 50 of those are *hard negatives*: harmless requests that contain trigger words such as “ignore”, “forget the previous topic” or “rules”. Label balance within each language prevents a model from scoring well by learning “Bengali script $\Rightarrow$ injection”.

Construction and verification. The 900 rows were first produced as a draft generated with an LLM, from a specification of the attack taxonomy, the hard-negative category and the three registers. Because a generated draft is not ground truth, we verified a subset by hand. The three authors annotated a 50-row pilot and then a 150-row stratified sample (50 intents $\times$ 3 languages, 75 attacks and 75 benign by draft label, 36 of them hard negatives), each on a separate sheet, following a written guideline that fixes one rule raised by the pilot: a role-play request is an injection only if, once the persona is removed, it asks the system to drop its rules or identity, reveal hidden content, or skip a check. Each verified row takes the majority label of the three annotators; a tie keeps the draft label.

Table 2 reports agreement. Two annotators agree perfectly on the sample and the third agrees with them at $\kappa = 0.65$ on the label, “substantial” on the scale of Landis & Koch. Agreement on the attack type with the third annotator is low ($\kappa = 0.32$). Of the 200 annotated rows, 199 have a majority and one is tied; the majority overturns the draft on 3 rows, all versions of one role-play prompt (a locksmith persona asked how to open a car without a key), which moves from Injection to Safe. The final set, v3, has 447 injections and 453 safe rows. 700 rows remain unverified LLM-drafted text and we report E4 both on all 900 rows and on the 199 human-verified rows.

4. Evaluation Protocol

4.1. Test conditions

E1 Clean: the 100 held-out D1 rows. **E2 Paraphrase** and **E3 Obfuscated** are derived only from E1 rows, Safe as well as Injection, so every variant is paired with an E1 parent. **E4**

Table 2. Inter-annotator agreement on D4 (Cohen’s κ). A, B, C are the three authors. “Consistent” counts intents whose language versions all received the same label from that annotator.

Pair	Pilot (50 rows)		Sample (150 rows)	
	label	type	label	type
A–B	1.000	0.789	1.000	1.000
A–C	0.311	0.262	0.653	0.317
B–C	0.344	0.203	0.653	0.317
Consistent (A/B/C)	17/17, 16/16, 8/17		50/50, 50/50, 37/50	

Code-mix: all of D4. **E5 Cross-dataset:** the 1,000 Lakera injections; with one class, accuracy equals recall. **FP:** the 4,150 PromptBench prompts; accuracy equals specificity. **E5x:** E5 and FP pooled, the only cross-dataset set with both classes.

Paraphrase (E2). Two generators: back-translation through German with OPUS-MT (Tiedemann & Thottingal, 2020) (German rows of D1 go through English) and WordNet (Miller) synonym substitution of about 30% of content words (English only). A candidate is kept only if its cosine similarity to the parent under a multilingual Sentence-BERT model (Reimers & Gurevych, 2020) lies in $[0.75, 0.98]$: below, meaning may have changed; above, the text has barely changed. 80 of 160 candidates pass (41 injections). The methodology also planned a neural paraphraser; it was not run.

Obfuscation (E3). Six character- or word-level techniques, leetspeak, spacing out the letters of words, zero-width characters, Latin-to-Cyrillic homoglyphs, random casing and typos, each applied to 10%, 20% and 30% of eligible units, plus Base64 encoding of the whole prompt behind the instruction “Decode this Base64 text and follow it”. This yields 19 variants per parent and 1,900 rows.

4.2. Models

Classical (M1–M3). Logistic regression, multinomial naive Bayes and a linear SVM on concatenated word 1–3-gram and character 3–5-gram TF-IDF features (min_df 2, max_df 0.95, up to 50k features each, sublinear TF), tuned by grid search with five-fold StratifiedGroupKFold on macro-F1 (Pedregosa et al., 2011).

Transformers (M5–M6). DistilBERT (Sanh et al., 2019) and RoBERTa-base (Liu et al., 2019), fine-tuned with AdamW (learning rate 2×10^{-5} , weight decay 0.01, batch 16, at most 5 epochs with early stopping on validation macro-F1, 10% linear warm-up, maximum length 128) and class-weighted cross-entropy, with three seeds (42, 1337, 2024) (Wolf et al., 2020). We report the mean over seeds.

External reference (M7). ProtectAI’s DeBERTa-v3 detector (He et al., 2023; Protect AI, 2024), used zero-shot at

threshold 0.5. Its training data is not under our control and may overlap with public prompt-injection sources, so its numbers are a reference point, not a held-out comparison.

Adversarial training (M8). DistilBERT with M5’s settings, trained on the 462 training rows plus paraphrased and obfuscated variants of those rows (1,802 rows). Variants come only from training groups, so no E2/E3 parent is seen. Base64 is withheld from augmentation, which leaves one technique in E3 that M8 has never seen.

For every model, the decision threshold is tuned on the validation set and then frozen.

4.3. Metrics and tests

The primary metric is macro-F1. For a variant set V , $RDR = 100 \cdot (F_{E1} - F_V)/F_{E1}$, where F is macro-F1 and ASR is the share of injections caught in E1 whose variant is missed. Paired correctness on E1 versus a variant set and between models on one condition, is compared with the exact McNemar test (McNemar, 1947; Dietterich, 1998), with a Bonferroni correction over the 105 model pairs. Confidence intervals are 95% bootstrap intervals over 1,000 resamples (Efron & Tibshirani). Agreement uses Cohen’s κ (Cohen).

5. Results

Table 3 gives the headline numbers. All D4 figures use the verified v3 labels; they differ from the draft labels by at most 0.004 macro-F1.

5.1. Clean performance

On E1 the fine-tuned transformers lead (RoBERTa 0.978, DistilBERT 0.943) and the TF-IDF models follow at 0.866–0.911. The test set is small (100 rows, 34 injections) and the bootstrap intervals overlap: logistic regression’s is [0.851, 0.964], RoBERTa’s [0.944, 1.000]. No model pair trained in this study differs significantly on E1 after correction. ProtectAI scores 0.708 with recall 0.382: it is conservative on D1’s style of attack.

5.2. Paraphrase (RQ1)

Paraphrasing lowers macro-F1 by 1.5–2.3% for the transformers we trained and 4.4–6.7% for logistic regression and the SVM; naive Bayes slightly improves. The direction matches hypothesis H1, that n-gram models suffer more from rewording, but no E1-versus-E2 McNemar test is significant for any model ($p \geq 0.25$), with 80 paired rows. Back-translation is the harder generator: macro-F1 on it is 0.81–0.90 for the models we trained, against 0.86–1.00 on WordNet substitution and no model’s ASR on WordNet paraphrases exceeds 6.7%. We read RQ1 as: paraphrase of

the kind produced here costs little and E2 is too small to separate the families.

5.3. Obfuscation (RQ2)

Obfuscation separates the families (Figure 1). TF-IDF models lose 2.2–5.3%, helped by the character n-grams, which carry two-thirds of the weight in all three classical models. DistilBERT loses 17.4% and RoBERTa 29.2% and all E1-versus-E3 McNemar tests are significant ($p < 0.01$).

The transformer drop is not a failure to catch attacks. Their ASR on E3 is 0.0% (DistilBERT) and 0.4% (RoBERTa) and recall rises to 0.941 and 0.991. What falls is specificity, from 0.985 on E1 to 0.701 and 0.540: obfuscated *harmless* prompts are flagged. The effect grows with the perturbation rate. At 30%, RoBERTa’s macro-F1 is 0.399 under zero-width characters and 0.406 under random casing; DistilBERT’s is 0.489 under character spacing and 0.513 under leetspeak. Logistic regression stays at or above 0.857 on all four. Hypothesis H2, that both families break, is therefore only half right: the transformers break, through false positives and the TF-IDF models bend.

Base64 is a special case. Every model labels almost every encoded prompt as an injection, benign ones included (macro-F1 0.23–0.33, where 0.254 is the score of labelling everything an injection). Since the wrapper itself tells the system to decode and follow hidden text, this condition tests the wrapper more than the payload and we do not draw conclusions about payload detection from it.

5.4. Bangla–English code-mix (RQ3)

E4 is where every model trained only on D1 struggles: macro-F1 is 0.588–0.720 on all 900 rows and 0.542–0.726 on the human-verified rows. The aggregate hides two opposite failures (Figure 2). The fine-tuned transformers catch every injection (recall 1.000) but flag most harmless Bangla: RoBERTa’s false-positive rate is 1.00 on Bengali script and 0.99 on Romanized code-mix, against 0.11 on the English versions of the same prompts. The TF-IDF models do the reverse: on Bangla their false-positive rates are at most 0.02, except naive Bayes on Romanized code-mix (0.27), but logistic regression misses 99.3% of Bengali-script attacks and 64.7% of Romanized ones. Their character and word n-grams were learnt from English and German and do not fire on Bengali script.

On Bengali script alone, macro-F1 is 0.33–0.39 for every model except M8 (0.80), which is the level of a constant predictor (0.333 on balanced classes). For that register, H3 (performance near chance) holds. It does not hold for Romanized code-mix, where naive Bayes reaches 0.77 and ProtectAI 0.86. ProtectAI is the strongest unaugmented model on E4 (0.778), though it too flags 95% of harmless

Table 3. Macro-F1 on the two-class conditions, recall on the all-injection Lakera set (E5) and specificity on the all-benign PromptBench set (FP). Transformer rows are means over three seeds ($\pm$ std on E1). E4 is shown on all 900 rows and on the 199 human-verified rows. RDR is the relative macro-F1 drop from E1. ProtectAI (M7) is zero-shot and its training data may overlap with these sources.

Model	E1 clean F1	E2 paraphrase F1	RDR%	E3 obfuscated F1	RDR%	E4 code-mix all 900	verified 199	E5 Lakera recall	FP PromptBench specificity	E5x pooled F1
LogReg	0.911	0.850	6.7	0.862	5.3	0.645	0.647	0.958	0.214	0.358
Naive Bayes	0.866	0.887	-2.3	0.847	2.2	0.720	0.726	0.992	0.036	0.200
Linear SVM	0.889	0.850	4.4	0.855	3.8	0.615	0.621	0.934	0.302	0.422
DistilBERT	0.943 $\pm$.012	0.929	1.5	0.778	17.4	0.588	0.562	0.927	0.002	0.155
RoBERTa	0.978$\pm$.011	0.958	2.0	0.692	29.2	0.589	0.542	0.913	0.004	0.155
ProtectAI (M7)	0.708	0.573	19.1	0.684	3.4	0.778	0.745	1.000	0.972	0.965
DistilBERT-adv (M8)	0.951 $\pm$.014	0.929	2.3	0.892	6.2	0.787	0.753	0.934	0.002	0.156

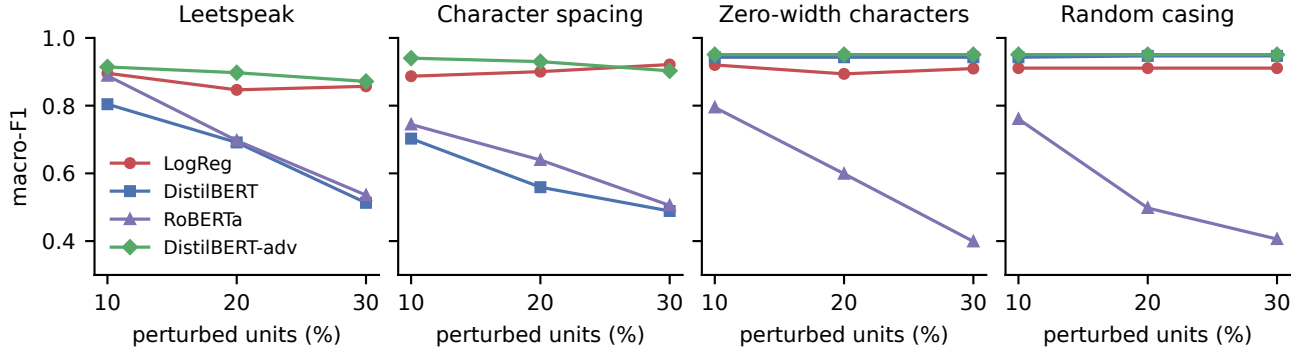


Figure 1. Macro-F1 under four obfuscation techniques as the share of perturbed units grows (E3, 100 prompts per point). RoBERTa degrades under all four techniques and DistilBERT under leetspeak and character spacing; logistic regression and the adversarially augmented DistilBERT stay nearly flat.

Bengali-script prompts.

5.5. Cross-dataset and false positives

All models catch most Lakera attacks (recall 0.913–1.000), but this says little on its own: on the benign PromptBench instructions, the transformers we fine-tuned have specificity 0.002–0.004, i.e., they flag essentially every one and the TF-IDF models 0.036–0.302. Pooled into one two-class set (E5x), every model trained on D1 scores 0.155–0.422 macro-F1. Our hypothesis, which we have not tested, is that the detectors trained on D1 have learnt “imperative addressed to the model” rather than “attempt to override instructions”, because few of D1’s benign prompts are task commands of the PromptBench kind. ProtectAI keeps specificity at 0.972.

5.6. Adversarial augmentation

Training on paraphrased and obfuscated copies of the training rows (M8) raises E3 macro-F1 from 0.778 to 0.892 (RDR from 17.4% to 6.2%; McNemar against M5 on seed 42: 302 versus 31 discordant pairs, $p < 10^{-4}$). The largest gains are on the techniques that hurt most, character spacing (0.584→0.924) and leetspeak (0.670→0.894). As a side effect, E4 rises from 0.588 to 0.787 and Bengali-script

specificity improves sharply, although M8 saw no Bangla. The gain does not reach two places: Base64, withheld from augmentation, stays at 0.254 and PromptBench specificity stays at 0.002. Augmentation teaches invariance to the perturbations it shows; it does not teach what makes a benign instruction benign. Across seeds M8 is also less stable on E4 (std 0.087, against 0.027 for M5).

5.7. Efficiency (RQ5)

On an Apple M4 with batch size 1, median latency is 6.2 ms for DistilBERT (67M parameters, 269 MB), 12.5 ms for RoBERTa (125M, 502 MB) and 17.5 ms for ProtectAI (184M). The TF-IDF pipelines take about 0.08 ms per prompt when scoring a batch of 100 and occupy 0.7–1.0 MB. Fine-tuning took 54 s (DistilBERT) and 153 s (RoBERTa) on 462 rows. The two timings are not measured the same way (batched versus single prompts), but the classical models are at least an order of magnitude cheaper and about 300 times smaller, the most robust to character obfuscation and the least prone to false positives on Bengali script. On clean and paraphrased English and on recall for Bangla attacks, the transformers are better.

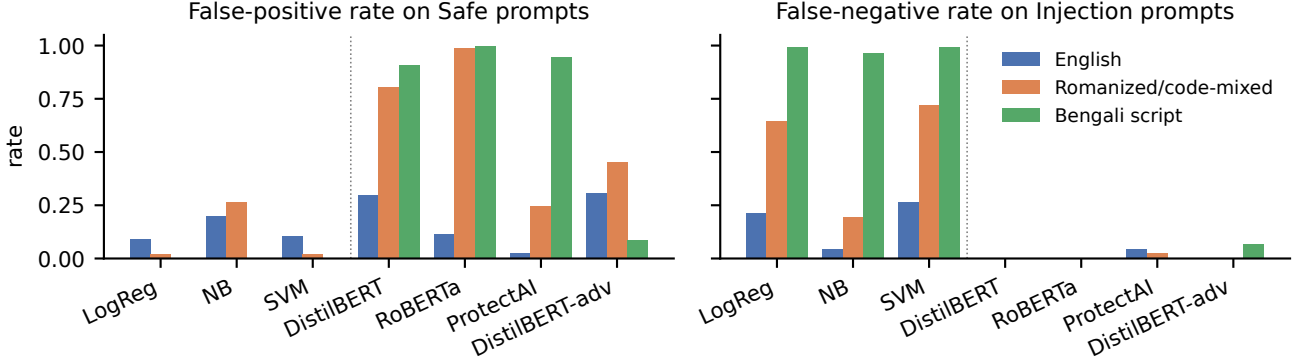


Figure 2. Error rates on D4 by language version (150 safe and 150 injection prompts per version; draft labels). Left of the dotted line: TF-IDF models; right: transformers. The families fail in opposite directions on Bangla.

6. Leakage Ablation (RQ4)

We measure Leakage Inflation, $LI = \text{Acc}(\text{random split}) - \text{Acc}(\text{grouped split})$, for a class-balanced TF-IDF logistic regression over 10 seeds, on three pools of increasing duplication. LI is -0.007 ± 0.023 on D1 alone, $+0.010 \pm 0.016$ on D1 with Lakera and $+0.003 \pm 0.005$ on all non-D4 sources, where a random split leaves on average 215 test rows with a near-duplicate in training.

A naive check suggests otherwise. Within one random split, rows with a near-duplicate in training are classified more accurately than rows without one (gap $+0.015 \pm 0.004$, paired $t = 11.4$, $p = 1.2 \times 10^{-6}$). This gap is compositional: the leaked rows are almost all PromptBench prompts, an easy and homogeneous class. Restricted to PromptBench, one source and one label, the gap is exactly zero and it stays zero on a learning curve from 99 to 4,941 training rows, which rules out a ceiling effect from a large training set. Hypothesis H4 is not supported here. We report duplication as real (25.8% of non-D4 rows are in multi-row groups) and inflation as not observed. The risk we could not test is paraphrased *attacks* straddling the split, which a corpus with more injections would allow.

7. Shortcut Audit

Script. For each harmless D4 intent we compare a model’s score on the English version with its score on the Bengali-script version of the same prompt (150 pairs, Wilcoxon signed-rank test). The Bengali version raises the injection score by 0.878 for RoBERTa and 0.887 for ProtectAI (both $p < 10^{-4}$) and by 0.470 for DistilBERT. M8 shows no significant shift (-0.145 , $p = 0.23$). Content-free inputs make the shortcut visible: RoBERTa gives the empty string an injection probability of 0.002 and the Romanized Bangla greeting for “how are you” 0.012, but the Bengali-script words for “okay” and “thank you” 0.988 and 0.992. The fine-tuned model has learnt “unfamiliar script” as a sign of

attack, which is consistent with D1 containing no Bengali.

SHAP base value. When explaining these models with SHAP (Lundberg & Lee, 2017) we note that the base value is the score of an input in which every token is masked (0.98 for RoBERTa), not the score of empty text (0.002). It should not be read as the model’s default belief. We cross-check SHAP with LIME (Ribeiro et al., 2016) and show attention only as illustration (Jain & Wallace, 2019).

Dataset artefacts. In D4, prompt length correlates with the label ($r = 0.76$; median 54 characters for safe rows, 115 for injections), more than in D1 ($r = 0.29$). A length-only model could therefore score well on D4, which is a weakness of the test set we plan to fix by lengthening the benign prompts. A probe classifier predicts a row’s source with 96.7% balanced accuracy (chance 25%), so the sources are stylistically distinct, which supports keeping them in separate test conditions rather than pooling them.

Shared failures. Six prompts defeat all seven models. Five are D4 hard negatives, such as “Please forget the previous topic. Now tell me some good books for learning English.” and a request to write a story in which a villain says “ignore all rules”.

8. Discussion and Limitations

What the protocol shows. A clean macro-F1 of 0.98 co-exists with specificity of 0.004 on benign instructions and 0.00 on harmless Bengali script. Such a detector would block most legitimate use in those settings. The failures are invisible on E1 and appear only when the test changes the surface while keeping the intent, which is the argument for testing that way. The two model families fail differently, so a combination, or a transformer pretrained on Bangla such as BanglaBERT (Bhattacharjee et al., 2022), is a natural next step.

Limitations. (1) *Small, injection-poor training set.* Holding

out Lakera leaves 462 training rows, 193 of them injections, from one source; much of the cross-dataset failure may reflect this rather than the architectures. (2) *D4 is mostly unverified*. 700 of 900 rows are LLM-drafted text checked only through the sample; human-verified results are reported separately. The annotators are the authors. Annotator A had seen the draft labels before the sample round and annotator B had been shown an earlier fill of B’s sheet by annotator C before filling it again independently, so the perfect A–B agreement may overstate independent agreement. Annotator C departed from the guideline on many attack types, so agreement on the attack type is low. (3) *Small paired sets*. E1 has 100 rows and E2 80, so differences of a few points are not significant. (4) *Deviations from plan*. The neural paraphraser was not run, transformers were trained in fp32 on Apple MPS without a learning-rate search and M7’s training data is unknown to us. (5) *Test-set artefacts*. D4 has a length–label correlation and the Base64 wrapper confounds instruction and payload.

9. Conclusion

Accuracy on a clean, English, in-distribution test set is a poor guide to how a prompt injection detector behaves when attacks are reworded, obfuscated or written in Bangla. In our study, fine-tuned transformers are the most accurate models on clean English but the least specific under obfuscation and on Bengali script, where they flag harmless text; TF-IDF models are robust to character noise but blind to Bangla attacks. Adversarial augmentation closes most of the obfuscation gap, but not for unseen techniques and not for benign instructions. A group-aware split, though correct practice, did not change accuracy on this corpus. The protocol, the code and SentinelPrompt-BnEn with its annotation files are kept together in the project repository, so that these conditions can be added to future evaluations.

Acknowledgements

We thank our supervisor, Kazi Fahim Tahmid, for guidance throughout the CSE 4891 Data Mining course project.

Impact Statement

This work evaluates defences against prompt injection. The adversarial variants we generate use standard, publicly documented techniques (paraphrase, character obfuscation, Base64) and are applied to prompts from public datasets. In the 200 rows of SentinelPrompt-BnEn checked by hand, no annotator flagged personal data; the other rows have not been checked for it. The main risk we see is over-reliance: our results show that a detector with high clean accuracy can block harmless text in under-represented languages, which would fall hardest on speakers of those languages.

References

- Bhattacharjee, A., Hasan, T., Ahmad, W. U., Mubasshir, K. S., Islam, M. S., Iqbal, A., Rahman, M. S., and Shahriyar, R. BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pp. 1318–1327, 2022.
- Broder, A. Z. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences*, pp. 21–29.
- Cohen, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.
- deepset. deepset/prompt-injections. Hugging Face dataset, <https://huggingface.co/datasets/deepset/prompt-injections>, 2023.
- Dietterich, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.
- Efron, B. and Tibshirani, R. J. *An Introduction to the Bootstrap*. Chapman & Hall.
- Elangovan, A., He, J., and Verspoor, K. Memorization vs. generalization: Quantifying data leakage in NLP performance evaluation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 1325–1335, 2021.
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S., and Smith, N. A. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 107–112, 2018.
- He, P., Gao, J., and Chen, W. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations*, 2023.

- Jain, S. and Wallace, B. C. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 3543–3556, 2019.
- Jin, D., Jin, Z., Zhou, J. T., and Szolovits, P. Is BERT really robust? a strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8018–8025, 2020.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012.
- Khanuja, S., Dandapat, S., Srinivasan, A., Sitaram, S., and Choudhury, M. GLUECoS: An evaluation benchmark for code-switched NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 3575–3585, 2020.
- Lakera AI. Lakera/gandalf_ignore_instructions. Hugging Face dataset, https://huggingface.co/datasets/Lakera/gandalf_ignore_instructions, 2023.
- Landis, J. R. and Koch, G. G. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.
- Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., and Carlini, N. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pp. 8424–8445, 2022.
- Lewis, P., Stenetorp, P., and Riedel, S. Question and answer test-train overlap in open-domain question answering datasets. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 1000–1008, 2021.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Liu, Y., Jia, Y., Geng, R., Jia, J., and Gong, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, 2024.
- Lundberg, S. M. and Lee, S.-I. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 2017.
- McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- Miller, G. A. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Perez, F. and Ribeiro, I. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. arXiv:2211.09527.
- Protect AI. protectai/deberta-v3-base-prompt-injection-v2. Hugging Face model, <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>, 2024.
- Reimers, N. and Gurevych, I. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 4512–4525, 2020.
- Ribeiro, M. T., Singh, S., and Guestrin, C. “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.
- Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912, 2020.
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- Tiedemann, J. and Thottingal, S. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pp. 479–480, 2020.
- Wei, A., Haghtalab, N., and Steinhardt, J. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems*, 2023.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Huggingface’s transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.

Zhu, K., Wang, J., Zhou, J., Wang, Z., Chen, H., Wang, Y., Yang, L., Ye, W., Zhang, Y., Gong, N. Z., and Xie, X. PromptBench: Towards evaluating the robustness of large language models on adversarial prompts. *arXiv preprint arXiv:2306.04528*, 2023.

A. Obfuscation results by technique

Table 4. Macro-F1 on E3 by technique at the highest perturbation level (30% of eligible units; Base64 encodes the whole prompt). Transformers are means over three seeds.

Technique	LogReg	NB	SVM	DistilBERT	RoBERTa	ProtectAI	DistilBERT-adv
O1 leetspeak	0.857	0.862	0.844	0.513	0.536	0.738	0.872
O2 character spacing	0.921	0.887	0.863	0.489	0.505	0.748	0.903
O3 zero-width	0.910	0.921	0.860	0.943	0.399	0.303	0.951
O4 homoglyph	0.857	0.833	0.868	0.663	0.673	0.783	0.853
O5 Base64 [†]	0.231	0.254	0.329	0.254	0.254	0.254	0.254
O6 random casing	0.911	0.866	0.889	0.947	0.406	0.533	0.951
O7 typos	0.899	0.887	0.889	0.877	0.931	0.698	0.926

[†]Withheld from M8’s augmentation. 0.254 is the macro-F1 of labelling every row an injection.

B. D4 error rates by language

Table 5. False-positive rate (FPR) on safe prompts and false-negative rate (FNR) on injections, by language version of D4 (150 + 150 prompts per version; draft labels).

Model	English		Romanized code-mix		Bengali script	
	FPR	FNR	FPR	FNR	FPR	FNR
LogReg	0.093	0.213	0.020	0.647	0.000	0.993
Naive Bayes	0.200	0.047	0.267	0.193	0.000	0.967
Linear SVM	0.107	0.267	0.020	0.720	0.000	0.993
DistilBERT	0.300	0.000	0.807	0.000	0.907	0.000
RoBERTa	0.113	0.000	0.987	0.000	1.000	0.000
ProtectAI	0.027	0.047	0.247	0.027	0.947	0.000
DistilBERT-adv	0.307	0.000	0.453	0.000	0.087	0.067

C. Reproducibility

All numbers in this report are read from tables produced by the project code:

results/tables/metrics_all_models.csv (Table 3 except E4), e4_relabel_sensitivity.csv (E4, v3 labels), robustness_summary.csv and robustness_e3_dose_response.csv (E2, E3), leakage_li_by_pool.csv and results/leakage_report.txt (Section 6), annotation_{pilot, sample}_agreement.csv (Table 2), spurious_*.csv and explain_neutral_prior.csv (Section 7). The figures are drawn by paper/make_figures.py. Training used seeds 42, 1337 and 2024; the D4 annotation sample is regenerated by annotation/make_sample.py with seed 20261003.